



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

TESI DI LAUREA MAGISTRALE IN
COMPUTER SCIENCE AND ENGINEERING
INGEGNERIA INFORMATICA

Binary Instrumentation for Tracking Edge Coverage in Fuzzing

Author:

Scandroglio Edoardo

Student ID:

216600

Advisor:

Prof. Name Surname

Academic Year:

2026-27

Dedicated to someone.

Abstract

...

Keywords: key, words, go, here

Abstract in lingua italiana

...

Parole chiave: qui, vanno, le, parole, chiave

Contents

Abstract	i
Abstract in lingua italiana	iii
Contents	v
1 Background and Motivation	1
1.1 Fuzzing Background	1
1.2 Binary Rewriting Background	4
1.3 ELF Loading and Relocations	7
1.3.1 ELF Structure	7
1.3.2 ELF Linking	8
1.3.3 ELF Loading	9
1.3.4 ELF Symbols and Relocations	11
1.4 Problem Statement	13
1.5 State of the Art	15
1.5.1 AFL Functioning	16
1.6 Goals and Challenges	19
2 Approach	21
2.1 Forkserver Insertion	21
2.2 Shared Memory Instrumentation	24
2.3 Instrumentation Points Search	26
2.4 Actual Instrumentation	28
Bibliography	31

List of Acronyms	35
List of Figures	37
List of Tables	39
List of Algorithms	41

1 | Background and Motivation

This chapter will focus on presenting the context of this thesis. In particular, in Section 1.1 the fundamentals of fuzzing will be discussed, with particular attention to coverage-guided grey-box fuzzing. In Section 1.2 the background on binary rewriting is discussed, along with its approaches and difficulties. In the next chapter will be presented the approach used in order to resolve the main problems regarding the solution discussed in this thesis. In order to do so some concept of binary loading must be presented (Section 1.3). Section 1.4 states the problem addressed by this work, and Section 1.5 reviews the state of the art in binary-only fuzzing, also focusing on a description of AFL/AFL++ internals. In the last section (Section 1.6) I will present the main goals and challenges of this thesis, and explain why the implemented tool differs from the state of the art.

1.1. Fuzzing Background

Fuzzing is nowadays a widely used approach for software testing and vulnerability assessment [17, 27]. Its main strength derives from the fact that it can rapidly produce various inputs to be fed into a target executable. In general, a fuzzer can be divided into the **Input Generator**, which is responsible for providing inputs to the **Executor** that executes them. During the execution, the **Defect Monitor** checks whether the executed input causes a crash of the target.

In particular, the Input Generator must choose which input to provide to the executor among all the inputs in the *Input Space*. A subset of the *Input Space* is represented by the *Valid Input Space*. This is a subset of the input space that specifically contains all the inputs that are well formed with respect to the application being tested. For example, if the fuzzed target is a PDF reader, a valid input is a PDF file. A good Input Generator must choose inputs that are also valid inputs, because only by doing this is there a probability of finding a vulnerability [27]. Hence the need for a precise input selection and mutation strategy arises, in order to discover as many edge cases as possible.

In order to do so, the Input Generator must obtain some information from the Executor. Depending on the amount of information the Input Generator has, fuzzing can be

classified into *White-box*, *Black-box* or *Grey-box*. In Black-box fuzzing, the fuzzer does not have any knowledge of the execution [19]. In White-box fuzzing, instead, the source code must always be available, and techniques such as Symbolic and Concolic execution are used [11]. In Grey-box fuzzing, instead, the inputs are chosen using, for example, **Code Coverage** information (i.e. the number of lines of code visited during execution of the target). This approach is called *Coverage-Guided Grey-box fuzzing* and is based on prioritizing the inputs that discover new sections of code. Since the source code is not always available, Code Coverage is not always a viable Coverage Measure. In general, the coverage measures can be of different granularity [10]. The three main options are:

- **Block Coverage:** Keeps track of the hit count for each Basic Block (BB). This approach is used by some popular fuzzers such as libFuzzer or honggfuzz [10]. This method is often too imprecise because it fails to capture the transitions between BBs.
- **Edge Coverage:** Keeps track of the hit count of every edge in the Control Flow Graph (CFG) (i.e. the transition between two BBs). This is the approach used by AFL and AFL++ [8]. It maintains overheads similar to Block Coverage but can track the transitions between BBs and not only their execution, adding some context information. However, it lacks precision in keeping track of the order in which the edges are taken.
- **Path Coverage:** This is the most complete metric and keeps track of the complete path taken during the execution of a specific input, thus also keeping track of the order in which the edges are taken. The main drawback of this metric is that a single execution path could become very large, resulting in large runtime and memory overheads.

The best approach to Code Coverage, in terms of overhead and effectiveness, is **Edge Coverage** [10], which keeps track of which edges in the CFG are taken during execution without the overheads incurred by keeping track of the entire execution path. Coverage information is obtained through the instrumentation of the target application. Depending on the availability of the source code, the instrumentation can be added in different ways:

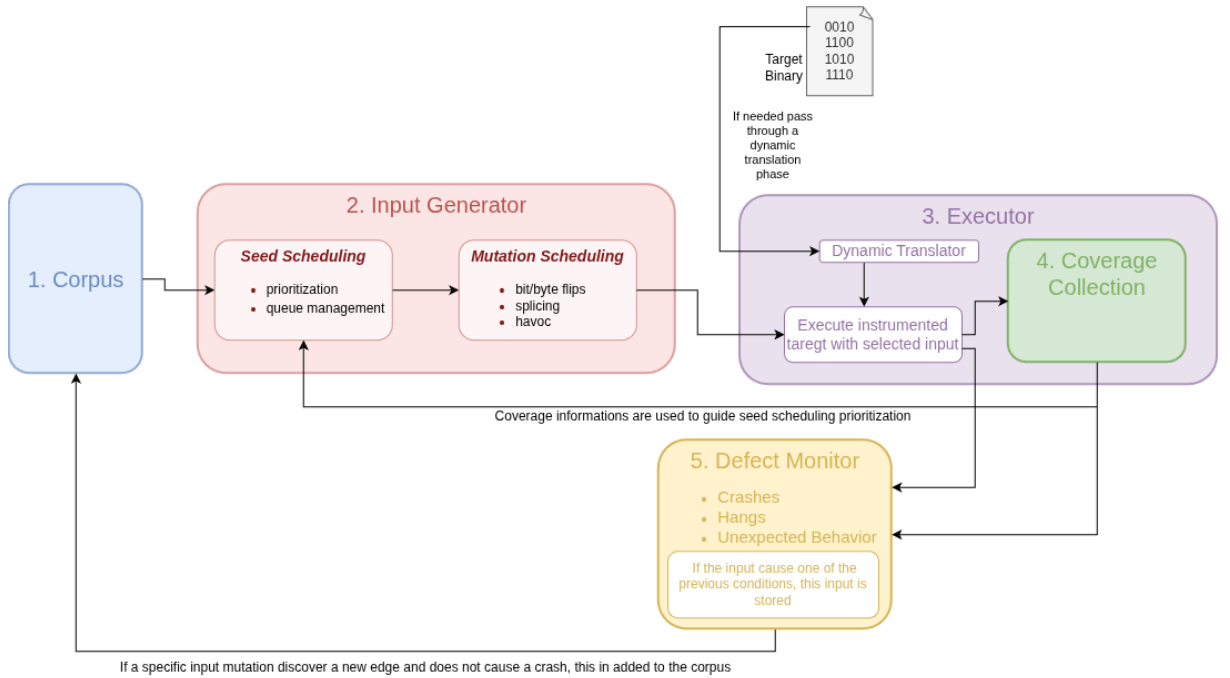
- The source code is available: In this case the instrumentation is added at *Compile Time*.
- The source code is not available: In this case there are two different techniques for instrumenting the application:
 - **Static Binary Instrumentation (SBI):** The instrumentation is added be-

fore the target is executed, without access to the source code, relying on Static Binary Rewriting techniques.

- **Dynamic Binary Instrumentation (DBI):** The instrumentation is added during program execution, relying on Dynamic Binary Rewriting techniques.

Regardless of the availability of the source code, the *Coverage-Guided Grey-box fuzzing* technique can be summarized as shown in fig. 1.1.

Figure 1.1: Generic diagram describing the Coverage-Guided Grey-box fuzzing workflow



Specifically for **Edge Coverage**, fig. 1.1 can be described as follows:

1. **Seed Corpus:** Represents the set of inputs on which mutations are computed. At the beginning it is composed of user-defined seeds, but during fuzzing this set can grow if a specific mutation discovers new edges not yet found.
2. **Input Generator:** Composed of two main parts:
 - **Seed Scheduling:** responsible for deciding which seed in the corpus to mutate.
 - **Mutation Scheduling:** responsible for deciding which mutation or mutations to apply to the selected seed.
3. **Executor:** The component that actually executes the target binary. If the target binary has already been instrumented with Edge Coverage tracking instructions, it is

executed as is; otherwise, in the case of DBI, a dynamic translator must dynamically instrument the binary.

4. **Coverage Collection:** In the case of Edge Coverage, this part is represented by the data structure used to keep track of which edges between BBs have been taken and which have not.

This information is used, in some tools such as AFL++ [8], to guide Seed Scheduling, a technique that, given different inputs with the same level of coverage (i.e. number of edges visited), prioritizes the input that is smaller in size and takes less execution time.

5. **Defect Monitor:** This component checks the exit status of the target to catch any crash or timeout of the target, or any other unexpected behavior. If none of these conditions are met, meaning that the target ends normally, the coverage is checked to see whether the current input discovered a new edge. If the input discovered a new edge, that input is added to the corpus.

1.2. Binary Rewriting Background

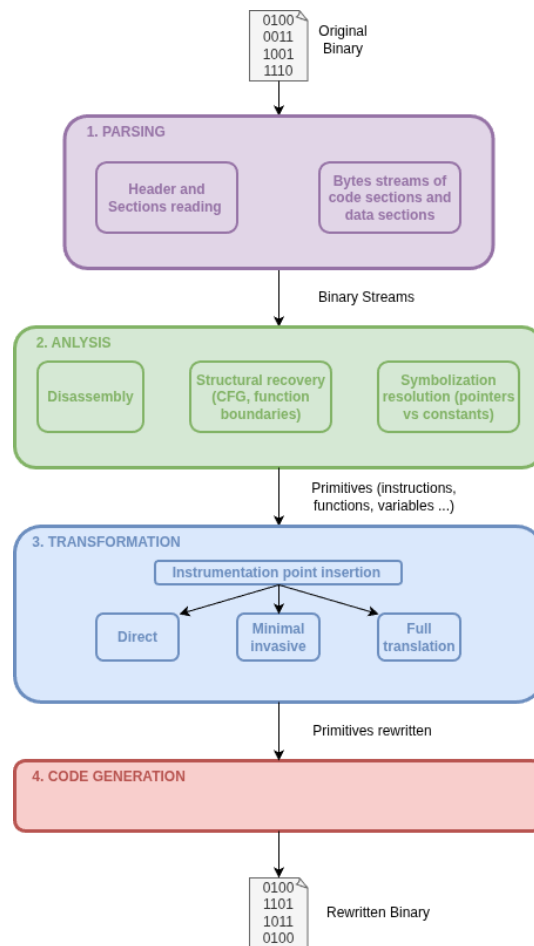
Binary rewriting consists in altering an already compiled executable without having access to the source code, while ensuring that the modified binary remains executable. The process of binary rewriting can be classified into Static Binary Rewriting, if the binary is stored in persistent memory while being rewritten, and Dynamic Binary Rewriting, if the binary is being executed. The process of binary rewriting can be divided into four main steps [23]:

1. *Parsing:* In this phase the main goal is to retrieve information about the binary. In particular, the output of this phase is a raw stream of instructions (in machine code) and data. This information is then used in the Analysis step.
2. *Analysis:* The information gained from the Parsing phase is analyzed to reconstruct the program structure. For example, the raw stream of machine code is passed through a disassembler that reconstructs the assembly instructions. A structural recovery is performed, which includes function detection and CFG reconstruction, and labels and symbols are extracted from the raw stream of data.
3. *Transformation:* Once the CFG has been reconstructed and function and data boundaries have been recovered, the binary rewriter can modify the binary at the Instrumentation Point (IP). These are user-defined locations in the binary where the rewriter must add, modify or delete an instruction. The IP can be coarse- or

fine-grained. In the first case, for example, the instrumentation could be inserted at branch level (at the beginning of each BB). In the second case, instead, any instruction could potentially be modified.

4. *Code Generation*: In this last step, the changes made in the previous steps must be integrated into the binary while preserving its executability. This can be done in different ways. For example, the changes could be inserted into a new section of the executable, redirecting execution to the new code at the IP. Alternatively, this step could also be performed by reassembling a completely new executable starting from the changes made at the IP. This second approach is much harder, because it requires preserving all the references inside the executable and preserving the executable layout.

Figure 1.2: An example of a Static Binary Rewriting Workflow

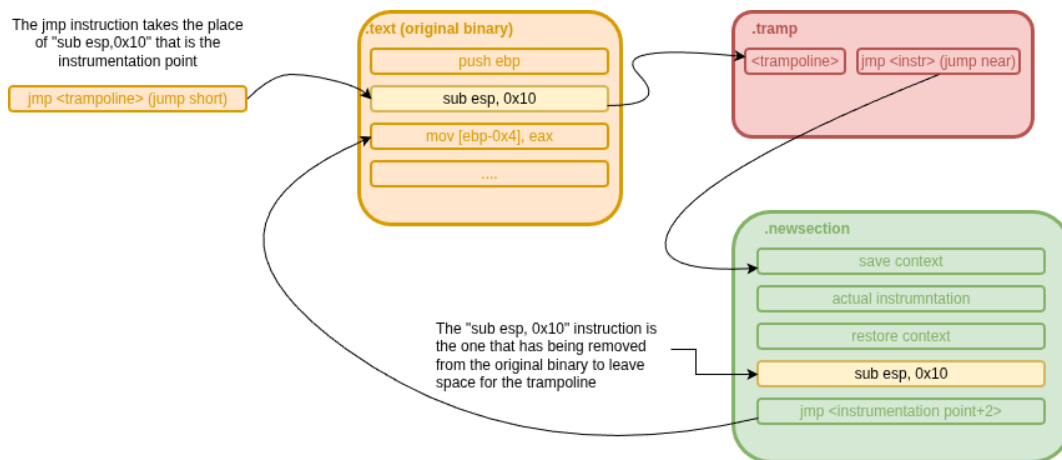


In fig. 1.2, a general scheme for static binary rewriting can be seen. In this case all the phases occur prior to execution, so the cost in terms of overhead is paid only once, before the execution. The main problems are disambiguating between code and data (i.e.

deciding whether a stream of bytes represents machine code instructions or constants placed inline in the .text section) and resolving indirect branches [3, 23]. This problem is generally undecidable and can only be solved using heuristics. There are three main families of static transformation:

- **Direct:** Consists in directly modifying the instruction. Works well with RISC systems with instructions having the same length. Generally fails when adding or removing instructions, because this does not preserve the offset into the .text section.
- **Minimal-invasive:** The new code is inserted into a new section of the binary, and at the IP an unconditional jump to the new section is placed. The main limitation of this approach arises when the new section is far from the IP and a near jump (necessary to reach the new section) uses a different number of bytes than the instruction at the instrumentation point. In fact, in x86-64 assembly a short jump uses 2 bytes while a near jump uses up to 5 bytes. So, if a short jump is needed at the instrumentation point for alignment reasons, but the new section is too far away and a near jump is needed instead, it is necessary to add a **trampoline**. This approach can be seen in fig. 1.3. Trampoline-based instrumentation is one of the most common approaches to binary rewriting [23].
- **Full-translation:** The binary is lifted into an Intermediate Representation (IR) that makes it possible to add an IP at instruction level. The IR is then reassembled into machine code after the changes are made.

Figure 1.3: An example of Static Instrumentation using the trampoline technique



With regard to Dynamic Binary Rewriting, the Analysis and Transformation phases are executed at runtime for each instruction. In this case, none of the problems that arise in Static Binary Rewriting occur, since the binary is being executed, so indirect branches can

be resolved using the values in the registers, and there is no need to disambiguate between constants inside the `.text` section and instructions. The main drawback of this approach is the prohibitive overhead caused by the dynamic analysis and transformation of each instruction, together with the fact that only the taken branches can be modified. In fact, if a branch is not actually taken during execution, its instructions never reach the rewriter.

A specific type of binary rewriting is Binary Instrumentation. Binary instrumentation uses binary rewriting techniques to add instructions that track binary behaviour, log events, or, in the case of this thesis, insert edge coverage tracing instructions.

1.3. ELF Loading and Relocations

Executable and Linkable Format (ELF) is the standard executable format under Linux operating systems. The work done in this thesis heavily involves the ELF structure, with particular focus on some sections used for relocation, alongside the process of executing an ELF.

1.3.1. ELF Structure

In this subsection the process of ELF execution is discussed, with a focus on the parts of the execution flow that will influence this work.

The **ELF Header** is a structure located at the beginning of the ELF and contains some important information about the type of ELF, the target operating system Alongside this information there is the location of two important data structures inside the ELF:

- The **Program Header Table**, which defines the *Execution View*
- The **Section Header Table**, which defines the *Linking View*

The **Program Header Table** contains information needed to create a runtime image of the program in memory, via the **Segments** (*i.e. which parts of the file must be loaded into memory, which shared objects are needed . . .*). Each segment contains the access flags for that part of memory (*i.e. Read/Write/Execute*). The **Section Header Table** contains information about the **Sections** that compose the binary, together with their mapping to the segments (*e.g. which segment the `.text` section is mapped to*). Among all the sections, some contain specific structures used to perform specific tasks:

- **.dynamic**: *The dynamic section contains information used by the Dynamic Linker (e.g. the dependencies, the interpreter location . . .).*

- **.dynsym**: List of symbols defined and used by the binary. If the binary is not stripped, there is also the **.symtab** section, which contains a more complete list of symbols.
- **.hash** or **.gnu.hash**: Structures used for efficient symbol lookup during symbol resolution¹.
- **.rela.dyn** and **.rela.plt**: Both are relocation tables. The first is the standard one, the second is used in the case of **Lazy Binding**.
- **.shstrtab**, **.dynstr** and **.strtab**: Hold lists of strings referenced by other parts of the ELF. The first holds section names, the second holds the names of the symbols contained in the **.dynsym** section, and the last one holds the names of the entire **.symtab** section.
- **.got** and **.got.plt**: Contain the Global Offset Table (GOT), filled by the dynamic linker with the addresses of external functions and variables. The first is the standard one, the second is used by the Procedure Linkage Table (PLT).
- **.plt**: Contains the PLT, which is an executable section made up of trampolines. In particular, it contains stubs for functions that might be located in another ELF or Shared Object (SO). The stub references the corresponding GOT entry containing the resolved address of the function. This section is used in the case of **Lazy Binding**.
- **.init** and **.init_array**: Sections used for initialization. **.init** contains initialization code, while **.init_array** contains the addresses of initialization functions.
- **.fini** and **.fini_array**: Sections used for termination handling. **.fini** contains termination code, while **.fini_array** contains the addresses of termination functions.

1.3.2. ELF Linking

Before actually digging deep into ELF loading, the linking process must be reviewed. The **Linking phase** consists of joining together different pieces of code belonging to different files. For example, if a program uses the **printf** function, it needs to take the code of **printf** from the **libc** library. This process of joining together the executable that uses **printf** with the **printf** code located in another file is called **Linking**. Linking can be of two different types:

- **Static linking**: The needed code (e.g. the instructions of **printf**) is copied inside the binary at *Link Time*. This causes the size of the binary to be bigger (because

¹The **.gnu.hash** section is the more efficient version of **.hash**.

the code of external functions actually lies inside it) and consequently wastes RAM space (if 10 binaries all use the `printf` function, the code of `printf` is copied inside all 10 binaries).

- **Dynamic linking:** The needed code is not copied inside the binary; instead, the binary contains information (i.e. the address of the external function or variable) used to locate the needed resource. The resolution happens at runtime, when the Dynamic Linker finds and loads the library into the process's virtual address space and resolves the references to the symbols. This allows different processes to share the same SO.

Since nowadays the most commonly used approach is **Dynamic Linking**, the following discussion will focus on it.

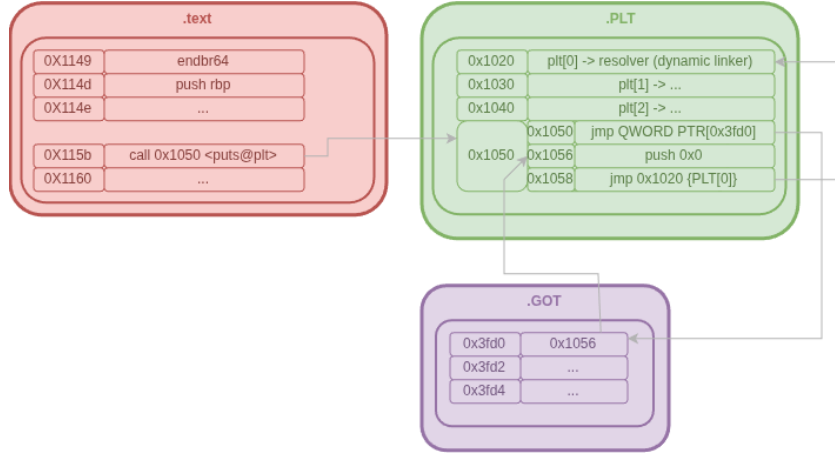
As stated before, the linking process can be used for both external variables and external functions. In both cases the process is the same, since both functions and variables can be exported as symbols.

When dealing with functions there is the possibility of using **Lazy Binding**. This technique consists of resolving the symbol only during the first invocation of the function, rather than at load time as happens with variables. In order to achieve **Lazy Binding** a stub section is used (i.e. `.plt`). When a program uses a library function, a PLT entry is added to the `.plt` section. As can be seen in fig. 1.4, the relocation is resolved lazily upon the first invocation of the function. When control is transferred to the corresponding PLT entry, the entry performs an indirect jump through its associated GOT entry. Before the symbol is resolved, this GOT entry contains an address that redirects execution back to the PLT, eventually reaching the PLT resolver stub. The resolver then invokes the dynamic linker, which performs the symbol lookup, resolves the relocation, and updates the GOT entry with the resolved address of the function, as can be seen in fig. 1.4b. Subsequent calls can therefore jump directly to the resolved address through the GOT.

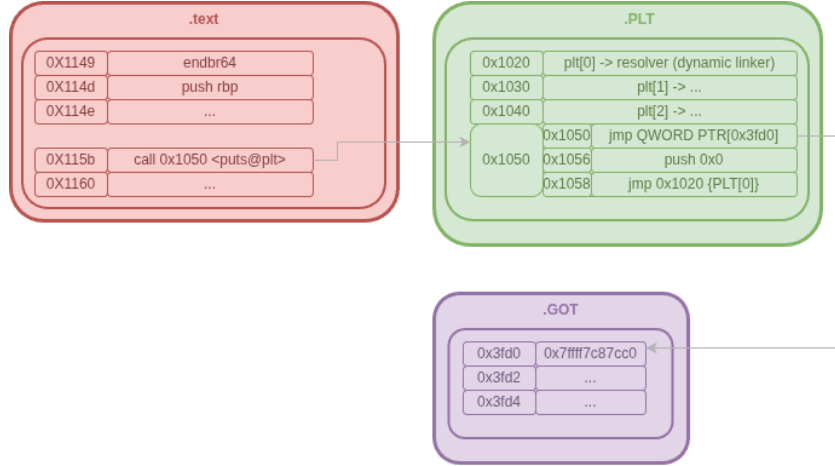
1.3.3. ELF Loading

The next useful step in order to understand the chosen approach consists of going into detail on the ELF Loading phase. There are 3 main actors that allow a binary to be executed properly:

- **Kernel:** responsible for the actual `execve` invocation and allocation of the different memory segments. Moreover, it is the one that executes the dynamic linker used to resolve dependencies prior to the execution of the binary.
- **Dynamic Linker:** Loads all the needed libraries into the process memory, resolves



(a) An example of puts function resolution using Dynamic Linking and Lazy Binding before the actual resolution



(b) Same binary as fig. 1.4a but with symbol resolved

Figure 1.4: An example of Lazy Binding involving .plt and .got

the external dependencies, prepares the .got and .plt, and executes the initializers of the libraries.

- **Target Binary:** Invoked by the Dynamic Linker, it prepares the environment for the main function and, in the end, runs it.

All these procedures can be seen in detail in fig. 1.5. The dynamic linker part is the one on which I will focus in this dissertation. The dynamic linker:

- *Loads all the DT_NEEDED entries:* these are the dependencies (i.e. the SOs needed by the binary). For example, if the binary uses an external function contained in *libc.so*, the library from which the function is exported must be added as a dependency.

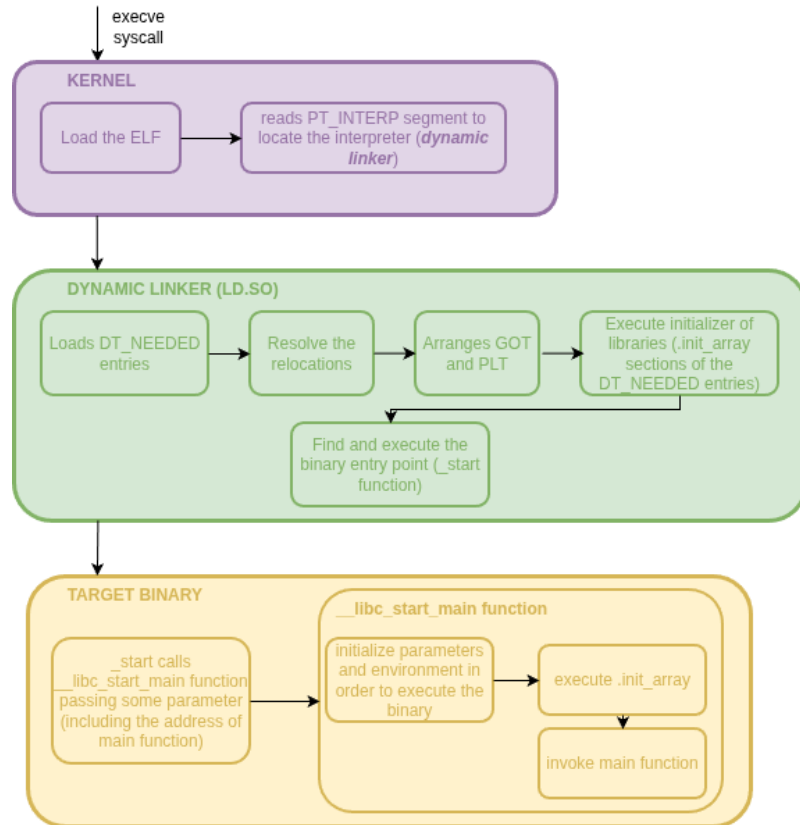


Figure 1.5: Dynamically linked ELF loading procedure

- *Resolves the relocations:* a focus on relocations will be presented in the next subsection.
- *Arranges the GOT and PLT.*
- *Executes all the initializer functions (constructors) for each dependency:* For each `DT_NEEDED` entry found by the dynamic linker, it executes all the functions contained in the `.init_array` section of that entry. These are functions declared with `__attribute__((constructor))`.
- *Finds and executes the binary entry point.*

1.3.4. ELF Symbols and Relocations

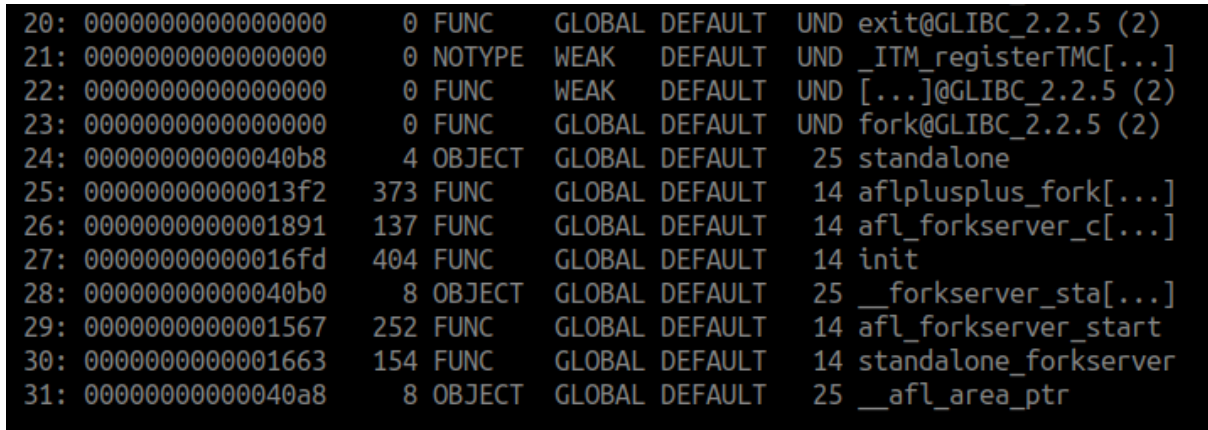
The dynamic linker uses symbols to link different ELF's together.

- A component (an ELF or a SO) can export a symbol representing a variable or a function inside it.
- Another component can reference the exported symbol and use it (if the first com-

ponent is a dependency of the second).

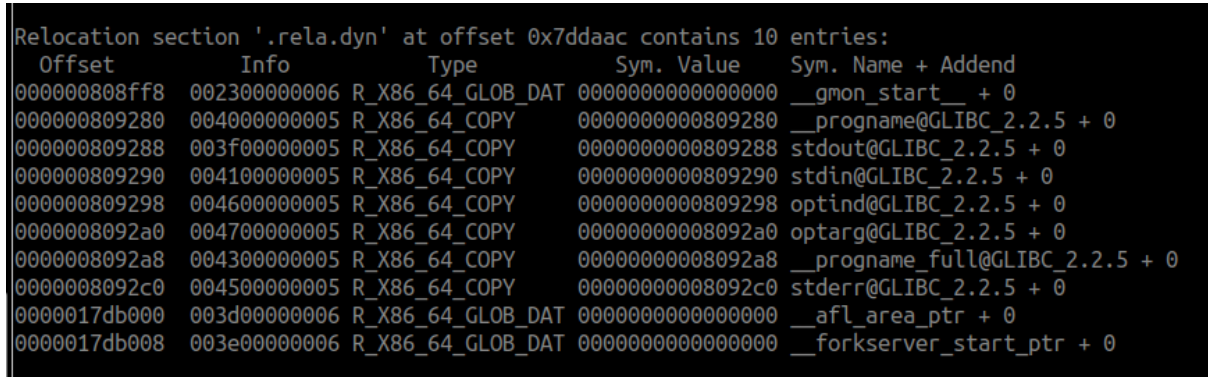
- The linker acts as the intermediary between the two components, responsible for matching the symbol names (the exported one and the referenced one) and storing the address of the exported symbol in the second component's address space in such a way that this component can access it.

All the symbols belonging to an ELF (both imported and exported) are stored in the *.dynsym* section. In fig. 1.6 a list of symbols extracted from a SO can be seen. The



20:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	exit@GLIBC_2.2.5 (2)
21:	0000000000000000	0	NOTYPE	WEAK	DEFAULT	UND	_ITM_registerTMC[...]
22:	0000000000000000	0	FUNC	WEAK	DEFAULT	UND	[...]@GLIBC_2.2.5 (2)
23:	0000000000000000	0	FUNC	GLOBAL	DEFAULT	UND	fork@GLIBC_2.2.5 (2)
24:	00000000000040b8	4	OBJECT	GLOBAL	DEFAULT	25	standalone
25:	00000000000013f2	373	FUNC	GLOBAL	DEFAULT	14	aflplusplus_fork[...]
26:	0000000000001891	137	FUNC	GLOBAL	DEFAULT	14	afl_forkserver_c[...]
27:	00000000000016fd	404	FUNC	GLOBAL	DEFAULT	14	init
28:	00000000000040b0	8	OBJECT	GLOBAL	DEFAULT	25	__forkserver_sta[...]
29:	0000000000001567	252	FUNC	GLOBAL	DEFAULT	14	afl_forkserver_start
30:	0000000000001663	154	FUNC	GLOBAL	DEFAULT	14	standalone_forkserver
31:	00000000000040a8	8	OBJECT	GLOBAL	DEFAULT	25	__afl_area_ptr

Figure 1.6: Example of a *.dynsym* table for a shared object



Offset	Info	Type	Sym. Value	Sym. Name + Addend
000000808ff8	002300000006	R_X86_64_GLOB_DAT	0000000000000000	__gmon_start__ + 0
000000809280	004000000005	R_X86_64_COPY	0000000000809280	__prognam@GLIBC_2.2.5 + 0
000000809288	003f00000005	R_X86_64_COPY	0000000000809288	stdout@GLIBC_2.2.5 + 0
000000809290	004100000005	R_X86_64_COPY	0000000000809290	stdin@GLIBC_2.2.5 + 0
000000809298	004600000005	R_X86_64_COPY	0000000000809298	optind@GLIBC_2.2.5 + 0
0000008092a0	004700000005	R_X86_64_COPY	00000000008092a0	optarg@GLIBC_2.2.5 + 0
0000008092a8	004300000005	R_X86_64_COPY	00000000008092a8	__prognam_full@GLIBC_2.2.5 + 0
0000008092c0	004500000005	R_X86_64_COPY	00000000008092c0	stderr@GLIBC_2.2.5 + 0
00000017db000	003d00000006	R_X86_64_GLOB_DAT	0000000000000000	__afl_area_ptr + 0
00000017db008	003e00000006	R_X86_64_GLOB_DAT	0000000000000000	__forkserver_start_ptr + 0

Figure 1.7: Example of a *.rela.dyn* relocation table extracted from a binary

first 4 symbols are external symbols that must be resolved. This can be seen because the Value column is 0 and Ndx is UND (undefined). The last 7, instead, are symbols declared inside the shared object. When the visibility is set to *DEFAULT*, this means that the symbol can be visible outside the shared object.

The process of resolving symbol references is called **Relocation**. The relocation algorithm is shown in algorithm 1.1. Relocating a symbol means resolving the symbol (i.e.

finding the address of that symbol in the original object), and writing that address inside the binary that has the relocation. Specifically, the symbol address is written inside the GOT. In fig. 1.7, the address at which the symbol address must be written can be seen in the “Offset” column.

Algorithm 1.1 Relocation resolution at ELF load time

Require: $L = [obj_1, \dots, obj_n]$ {The objects that have a dependency (DT_NEEDED) must already be loaded in memory}

Ensure: for each $obj \in L$, the respective GOT entry contains the resolved runtime address of the external symbol.

```

1: for  $obj \in L$  do
2:    $Reloc\_table \leftarrow obj.GET\_SECTION(".rela.dyn")$ 
3:   for  $reloc \in Reloc\_table$  do
4:     if  $reloc.type = R\_*\_RELATIVE$  then
5:        $addr \leftarrow obj.loadBias + r.addend$  {The symbol does not need to be resolved.
        The address must be adjusted in order to enable ASLR}
6:     else
7:       if  $reloc.type \in \{R\_*\_GLOB\_DAT, R\_*\_JUMP\_SLOT\}$  then
8:          $sIdx \leftarrow reloc.symbol\_index$ 
9:          $symbolName \leftarrow obj.dynstr[obj.dynsym[sIdx].name]$  {The name of the
          symbol is read from .dynsym}
10:         $target \leftarrow RESOLVESYMBOL(symbolName, L, obj)$ 
11:        if  $target = NULL$  then
12:          error: undefined symbol  $symbolName$ 
13:        else
14:           $(defObj, defIdx) \leftarrow target$  {defObj is the object that exports the symbol,
           defIdx is the index in the .dynsym table of defObj}
15:           $addr \leftarrow defObj.loadBias + defObj.dynsym[defIdx].value$  {The address of the
           resolved symbol is computed as base_address (of the object that exports the
           symbol) plus the offset at which the symbol is stored}
16:        end if
17:      end if
18:    end if
19:    write  $addr$  in  $obj.got$  at offset  $reloc.offset$  {The destination address at which to
     write the resolved symbol address is specified in the relocation entry}
20:  end for
21: end for

```

1.4. Problem Statement

Coverage-Guided Grey-box fuzzing is one of the most effective techniques for automatic vulnerability discovery in software [6, 20], due to its ability to generate a wide range of inputs based on the feedback obtained through Code Coverage information, which guides

Algorithm 1.2 RESOLVESYMBOL function used by algorithm 1.1

Require: name *name*, object list *L*, object requesting the resolution *requester*
Ensure: pair (*obj*, *idx*) of the first symbol found, or NULL if the symbol has not been defined by any object.

```

1: for obj ∈ L {The order is: first the executable, then all the DT_NEEDED dependencies
   in order of definition} do
2:   idx ← GNULIBLOOKUP(name, obj.gnuHash, obj.dynsym, obj.dynstr) {As seen before,
   the hashes are used in order to speed up the symbol lookup}
3:   if idx ≠ NOT_FOUND && obj.dynsym[idx].st_shndx ≠ SHN_UNDEF then
4:     return (obj, idx)
5:   end if
6: end for
7: return NULL

```

the fuzzer towards unexplored regions of code and therefore helps it find hidden vulnerabilities. In fact, *Black-box* fuzzing lacks a precise mutation strategy based on execution monitoring, while *White-box* fuzzing incurs high overheads due to Symbolic or Concolic execution. Furthermore, the latter approach also requires the availability of the source code, which is not always available. So *Grey-box* fuzzing seems to be the most promising trade-off between precision of input mutation and overhead. However, *Grey-box* fuzzing also has its own drawbacks. As a matter of fact, the effectiveness of this approach strongly depends on the availability of the source code. In fact, both SBI and DBI have many limitations. The main limitation of DBI is the huge amount of overhead (higher than 600% in the case of AFL++ in QEMU mode [20]), which makes the exploration of a substantial part of the valid input space almost impossible. Instead, the main difficulty of SBI arises in modifying already compiled binaries. There are several problems regarding Binary Rewriting, such as Indirect Control Transfers (e.g. `call rax`), shifts in binary sections, or instructions that require rewriting the instructions that reference these sections, and many others, as seen in the previous Section 1.2.

For all these reasons, the preferred technique nowadays is *Compile-time Edge Coverage instrumentation* with fuzzers such as AFL++, LibAFL [9], or libFuzzer [15]. However, in practice, for reasons of industrial secrecy (i.e. proprietary software) or due to precompiled libraries, applications are often closed source, and so Compile-time Edge Coverage instrumentation becomes unusable.

In these scenarios, binary-only fuzzing is the only viable approach. Although SBI presents many challenges, it nonetheless proves to be the most fruitful area, given its low overheads [20]. In this thesis I will address the problem of statically instrumenting a binary with Edge Coverage instructions in a way that makes the instrumented binary compatible with AFL++, which has been chosen as the fuzzer due to its popularity and because it repre-

sents the current state of the art, implementing most of the recently discovered techniques demonstrated by Andrea Fioraldi et al. [8].

The instrumentation must add as little overhead as possible with respect to compile-time overheads and must preserve the correctness of the binary. This is important in fuzzing, because lower overhead means more executions per second, so a larger portion of the valid input space can be explored.

1.5. State of the Art

In this section I will present some of the current state-of-the-art solutions in binary-only grey-box fuzzing, distinguishing between SBI and DBI approaches.

- DBI

- **AFL++ in QEMU/FRIDA mode** [8]: AFL++ in QEMU mode uses QEMU in *User-mode Emulation*, which consists in translating guest machine code (i.e. guest BBs) into an intermediate representation called Translation Block (TB) and subsequently appending the coverage tracing instructions at the end of it. This approach is based on emulating the entire CPU using QEMU’s virtualized environment. As stated before, AFL in QEMU mode has overheads on the order of 600% with respect to compile-time instrumentation [20].

AFL++ in FRIDA mode [1], instead, is based on shared library injection, in which a Frida [21] component called **Stalker** intercepts any BB that is about to be executed, copies it into a **Code Cache** and dynamically instruments the copy with coverage tracing instructions. Execution control is then transferred to the copy, which is executed and then re-intercepted by **Stalker** at the end. Unlike QEMU mode, it does not emulate the CPU, but instead executes within the memory space of the target program. AFL++ reports that FRIDA mode is approximately 2 to 5 times slower than compile-time instrumentation [2].

- **DynamoRIO/Pin** [4, 16]: Both these approaches share the same basic principle, called **Just in-time (JIT) code caching**. This consists in taking control of the thread when the target application starts, and from that moment on, every BB is intercepted before it is executed and is copied into a private **Code Cache**. The instrumentation is then added to the copy, and control is transferred to the code cache. The instrumentation process is more or less identical to FRIDA’s Stalker mechanism, but is not as optimized. Also in this case, the implemented version uses AFL [26] as the fuzzer [13, 14].

- **SBI**

- **Retrowrite** [7]: This tool performs symbolic lifting on the code, replacing every reference discovered during disassembly with a symbolic label. Doing this allows instructions to be added without modifying the references, achieving reassembleable assembly. While RetroWrite can add ASan instrumentation, it cannot natively add AFL-style edge coverage instructions. To do so, the lifted assembly must be instrumented with the afl-gcc tool in order to produce the patched binary. Moreover, community-reported issues [18] document missing instrumentation points when RetroWrite’s symbolized output is compiled through afl-gcc, caused by a mismatch between RetroWrite’s label-naming convention and afl-as’s basic-block detection heuristics. Finally, RetroWrite only works with PIE, non-stripped binaries.
- **AFL-Dyninst** [12]: AFL-Dyninst employs Dyninst’s binary rewriting capabilities to instrument an existing executable by inserting callbacks at selected basic blocks. Dyninst’s instrumentation mechanism is based on code patching and trampoline redirection [5], which causes many context switches and state saving/restoring operations. Its performance is slightly better than DBI, but this approach still reaches 500% fuzzing overhead, as demonstrated by Stefan Nagy et al. [20].
- **ZAFL** [20]: This tool uses two main phases: the first consists in reconstructing an IR using the Zipr binary rewriter. The resulting IR is then used by ZAX to perform an optimization on the IR, find the instrumentation points, and finally instrument these points. ZAFL works by inserting calls to a function that updates the shared map at the beginning of each BB. Despite ZAFL’s many improvements, Jifan Xiao et al. [24] demonstrated that it still causes crashes in 28% of the binaries used in their cited paper.

Before proceeding to the next section, in which I will discuss the main goals of this thesis, I want to discuss the core functioning of AFL and AFL++, given its importance for understanding my approach.

1.5.1. AFL Functioning

[25] As previously stated, AFL and, in particular, AFL++ are among the most widely used grey-box fuzzing frameworks [8]. AFL++ represents the community-based continuation of the original AFL project. The core functioning is virtually the same and consists of a Shared Memory (SHM) region shared between the fuzzer and the target. The SHM

size can vary, but the default is 65536 bytes.

Each “cell” of the SHM contains the number of times a given edge in the CFG has been taken during the execution of the target. Since the SHM has 65536 “cells”, each “cell” can contain a number from 0 to 255. The “cell” position is computed with a specific hash function applied to the IDs of the two BBs that form the edge. The main Edge Tracing update rule is summarized in algorithm 1.3. Line line 31 of algorithm 1.3 is used to avoid

Algorithm 1.3 AFL-style SHM update rule

```

1:  $idx \leftarrow current\_bb\_id \oplus previous\_bb\_id$ 
2:  $shared\_map[idx] \leftarrow shared\_map[idx] + 1$ 
3:  $previous\_bb\_id \leftarrow current\_bb\_id \gg 1$ 

```

updating the same SHM location when an edge is traversed forward or backward. An example of this could be a simple **do-while** loop, as seen in algorithm 1.4 and fig. 1.8.

The main idea behind AFL is to instrument each basic block of a target application with

Algorithm 1.4 Do-While Example

```

repeat
  {Basic Block A}
until Permanence_Condition
  {Basic Block C}

```

Without line 31 of algorithm 1.3, the hash of Edge1 would result in $ID_A \oplus ID_B$, and the hash of Edge2 (if the permanence condition is not met) would result in $ID_B \oplus ID_A = ID_A \oplus ID_B$, due to the symmetry of XOR. But since the hash is the same, there would be a hash collision that would cause the same “cell” to be updated twice instead of once.

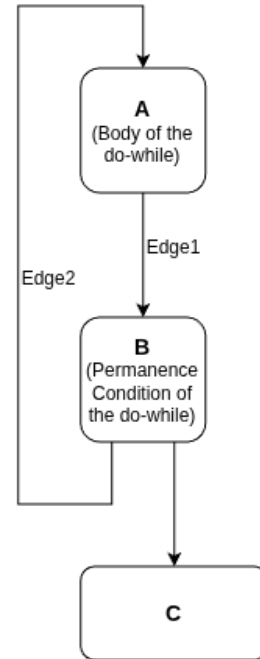


Figure 1.8: Do-while CFG

the shared map update rule. The fuzzer initializes the SHM memory with zeros, runs the instrumented target, which updates the SHM according to algorithm 1.3 until it reaches the end, and then returns to the fuzzer. The SHM ID is communicated to the target through an environment variable called `__AFL_SHM_ID`.

At this point the fuzzer checks the SHM it has just obtained, called `trace_bits`, searching for any newly discovered edge. This is done by keeping a structure called `virgin_bits`,

which contains all the edges not yet discovered and is initialized with ones. The comparison is performed through `trace_bits & virgin_bits`. If any position results in one after the logical AND, this means a new edge has been discovered. In this case, the `virgin_bits` data structure is updated and the input just executed is saved in the corpus. Otherwise, a new input is chosen, mutated and then executed.

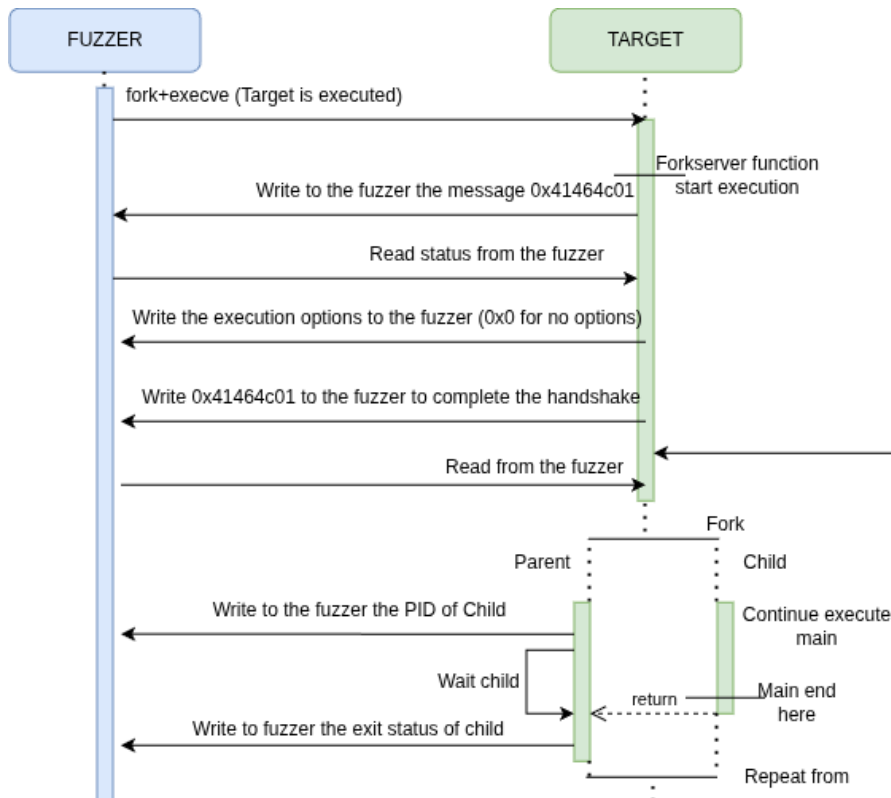
This approach requires a `fork()` call followed by an `execve()` call for each input, causing significant overhead for each input execution. Two main optimizations have been implemented:

- **forkserver**
- **persistent mode:** (available only in AFL++)

Forkserver

The forkserver makes it possible to avoid calling `execve` for every input, performing it only once at the start of execution. The forkserver is instrumented before the main function of the target application. The AFL++ forkserver protocol can be summarized as follows: ² The communication between the Fuzzer and the Target shown in fig. 1.9

Figure 1.9: AFL++ Forkserver Protocol



occurs unidirectionally through two different file descriptors created by the fuzzer:

- *FORKSRV_FD* (*fd 198*): used by the forklserver to read a message sent by the fuzzer.
- *FORKSRV_FD + 1* (*fd 199*): used by the forklserver to write a message to the fuzzer.

1. The forklserver sends the “Welcome” message to the fuzzer (0x41464c01).
2. The fuzzer replies with the status code. At this point the forklserver must check whether $status \wedge 0xffffffff = 0x41464c01$. If it is equal, execution continues; otherwise, the forklserver stops here.
3. The forklserver sends the execution options to the fuzzer. These can indicate that the target is running in persistent mode, or other optimizations.
4. The forklserver sends 0x41464c01 to the fuzzer to end the handshake.
5. The fuzzer writes a message to the forklserver (its content is discarded); the handshake is now finished. This message is the “go” signal for the forklserver.
6. The forklserver invokes `fork()`. The parent writes the PID of the child to the fuzzer and then invokes `waitpid()`.
The child continues execution of the main function until it ends.
7. When the parent returns from the `waitpid()` function (i.e. the child has terminated), the exit status of the child is sent to the fuzzer, and the forklserver waits for another “go” from the fuzzer.

1.6. Goals and Challenges

The main goal of this thesis is to develop a tool capable of inline, static edge-coverage instrumentation that preserves binary correctness while achieving performance comparable to compile-time instrumentation.

I have chosen to implement the SBI approach instead of DBI due to its low overhead. To do this, I will use the GRAPE binary rewriting framework, a novel approach for binary rewriting that preserves binary executability while introducing small overheads compared to a trampoline-based approach. This approach involves several challenges, mainly related to preserving the binary’s correctness and behavior. Moreover, the tool must be compat-

²The differences between the AFL forklserver protocol and the AFL++ forklserver protocol lie mainly in the messages exchanged between the forklserver and the fuzzer

ible with the already existing AFL++ fuzzer. This means implementing the forkserver communication protocol inside the target binary without recompiling it. This has created no small number of problems, mainly regarding the preservation of the ELF layout after adding new sections. Compared to the ZAFL approach, the SHM update instructions are placed directly inside the BB, rather than in a function called at each BB. This approach drastically increases the size of the file, causing many instruction repetitions (since the SHM instructions are always the same), but was chosen because it does not cause a context switch at each BB. In fact, in order to call a function, the state of the registers must be saved and restored. Moreover, the stack must be allocated, and the callee's RIP must be saved on the stack alongside the old base pointer of the stack. Performing all these operations at each BB would introduce significant overhead.

2 | Approach

In this chapter I will disclose the general aspects of the approach regarding the insertion of the Edge Tracing instructions inside an already compiled binary.

Four are three main steps to be fulfilled to have a binary that is able to trace Edge Coverage and to be compatible with AFL/AFL++:

1. Insertion of the forkingserver invocation prior the main (section 2.1).
2. Shared Memory address discovery (section 2.2).
3. Search of instrumentation points to trace Edge Coverage.
4. Instrument the previously found points with Edge Coverage tracing instructions.

2.1. Forkserver Insertion

This has been one of the most crucial points of the whole work. First of all I have tried to instrument the forkingserver function directly inline. This approach presents many problems. The biggest difficulty is the fact that a forkingserver needs some library functions to work properly:

- `read/write` to implement the communication protocol with the fuzzer.
- `fork/waitpid` to implement the forkingserver main logic.

These functions could be part of the target binary (if the binary already uses these functions) or not. In the best case, the binary already uses these functions and they are available. In this case, a call to these functions could be instrumented at any time, taking into consideration the type of linking with which the binary has been compiled.

- In case of *Static Linking*, the function instructions are already inside the target application, because the linker inserted them during compilation. So a call to these functions could be inserted by finding the function address and then instrumenting a call instruction to that address.
- In case of *Dynamic Linking*, the function instructions are not inside the target

application when it is launched, but the function addresses are dynamically resolved by the dynamic linker as needed. In this case, two new sections must be used (the `.plt` and `.got` sections, as seen in section 1.3).

If the functions required by the forklserver are part of the target binary, their calls can be easily instrumented. Depending on whether the binary is statically or dynamically linked, this can be done either by finding the address of the function and calling it directly, or by finding the address of its `.plt` entry and calling that, leaving the dynamic linker to resolve the function. The problem arises when the original binary does not include these functions. In this case, it is almost impossible to modify the `.plt` and `.got` sections to include the new functions while preserving the binary’s layout and correctness in the general case. In fact, in some exceptional cases padding may provide enough space in the `.got` section for a new entry, but this cannot be assumed in general. Therefore, this first approach has been completely discarded. Since the forklserver is more complex than a simple shared map update, due to the presence of many checks on environment variables and communication with the fuzzer, the best approach is to inject the compiled forklserver code into the target through *Shared Object Injection*. In order to achieve Shared Object Injection, the `.dynamic` section of the target binary must be modified. In particular:

- A `DT_NEEDED` entry is added with the name of the shared object to be injected.
1
- A `RUNPATH` entry is added containing the path where the actual shared object is saved.

2

The shared object injected into the target binary contains the code of the forklserver, which must be executed before the main of the binary. In order for a function of a shared object injected into a binary to be executed before the main, that function must be declared as a “constructor”. As stated in section 1.3, by declaring a function as a constructor, its address is placed inside the `.init_array` section and executed before the actual start of the executable’s main. When an executable has some dependencies, the dynamic linker executes the `.init` section and the functions contained in the `.init_array` section before actually starting the main of the executable.³ In fig. 2.1b the execution flow

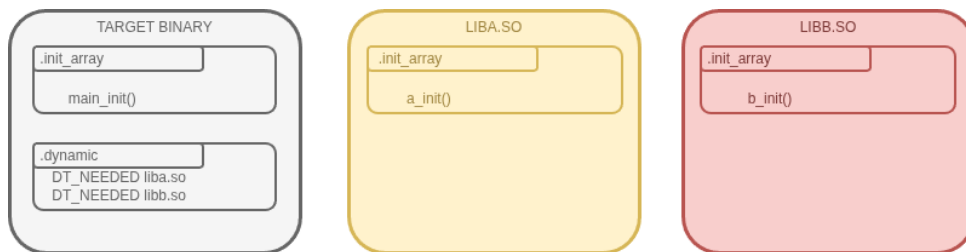
¹In reality the shared object name is a string that is written into the `.dynstr` section, and the `.dynamic` section specifies the offset of the string written into `.dynstr`.

²The injection could also be done with the `LD_PRELOAD` environment variable. This variable tells the dynamic linker to add the specified libraries as dependencies of the executable. The problem is that at each execution the variable must be reset with the name of the shared object. By adding the `DT_NEEDED` dependency instead, the dependency becomes persistent.

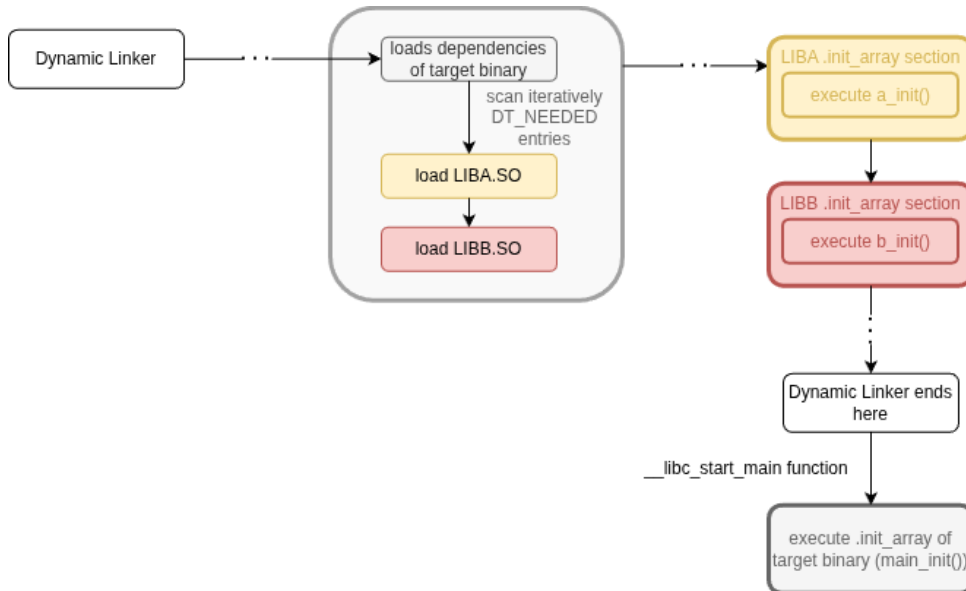
³The `.init` section is the legacy version of the `.init_array` section.

of the dynamic linker can be seen as it initializes the dynamic dependencies of a target binary that has the dynamic dependencies shown in fig. 2.1a.

1. The dynamic linker searches for *DT_NEEDED* entries of the target binary. When a dependency is found, it maps the library into a loaded Virtual Memory Area (VMA) of the process associated with the target binary being executed.
2. For each library found previously, the dynamic linker executes each function found in the *.init_array* section of that shared object.
3. When the dynamic linker finishes, the `_start` function is executed, and it in turn executes `__libc_start_main`, which executes all the functions in the *.init_array* of the target binary.



(a) Layout of a target binary with its dynamic dependencies



(b) Dynamic linker initialization of dynamic dependencies

Figure 2.1: An example of ELF execution with dynamic dependencies and a constructor function

Therefore, injecting a SO that has a function declared as constructor into the target binary could resolve the problem of instrumenting the forkserver so that it is executed

before the main.

2.2. Shared Memory Instrumentation

As stated in the previous chapter, the SHM is declared by the fuzzer and the forklserver can know the ID of the SHM thanks to an environment variable set by the fuzzer. The forklserver is injected as a constructor function of a SO for which a `DT_NEEDED` dependency has been added to the target binary. But, as stated before, the dynamic linker executes the constructor of the SO (and therefore the forklserver) before the actual initialization of the binary. Moreover, the SHM address is saved in a variable of the SO and is not, by default, visible inside the target binary. The best approach that allows a variable to be accessible outside a binary is to export the symbol that represents the variable.

As seen in section 1.3, the relocations for external symbols are resolved during the execution of the dynamic linker before the main (or at the first invocation of the function in the case of lazy binding). Since the SO contains the symbol holding the address of the SHM, the idea to let the target binary know the address of the SHM is:

- The SO exports that symbol, declaring it with `__attribute__((visibility("default")))` ⁴
- Add a relocation entry to the target binary.

By doing this, the dynamic linker, at binary loading, will resolve the relocation, finding the address of the variable that stores the address of the SHM inside the SO, and write it into the `.got` of the target binary.

Since, as stated in section 2.1, it is difficult to add a GOT entry in the general case (because it depends on the presence of available space in the GOT), the chosen approach has been to add a new section to the binary where the needed symbols, after relocation, will be written. Now the complete method to make the SHM accessible to the target instrumented binary can be reviewed:

1. Add a new section to the binary. This section will be needed as a GOT in which the resolved symbol values will be written.
2. Add a dynamic symbol to the binary. This is the local symbol that will be referenced by the relocation table. The name of the local symbol must be the same as the symbol exported by the SO that has already been injected as a dependency of the

⁴It is not mandatory to explicitly write the visibility attribute, because each symbol has “default” visibility by default.

binary.

3. Add a dynamic relocation entry. This relocation entry must link the symbol to be relocated with the memory address at which the resolved value will be written (i.e. the section previously created to host the GOT).

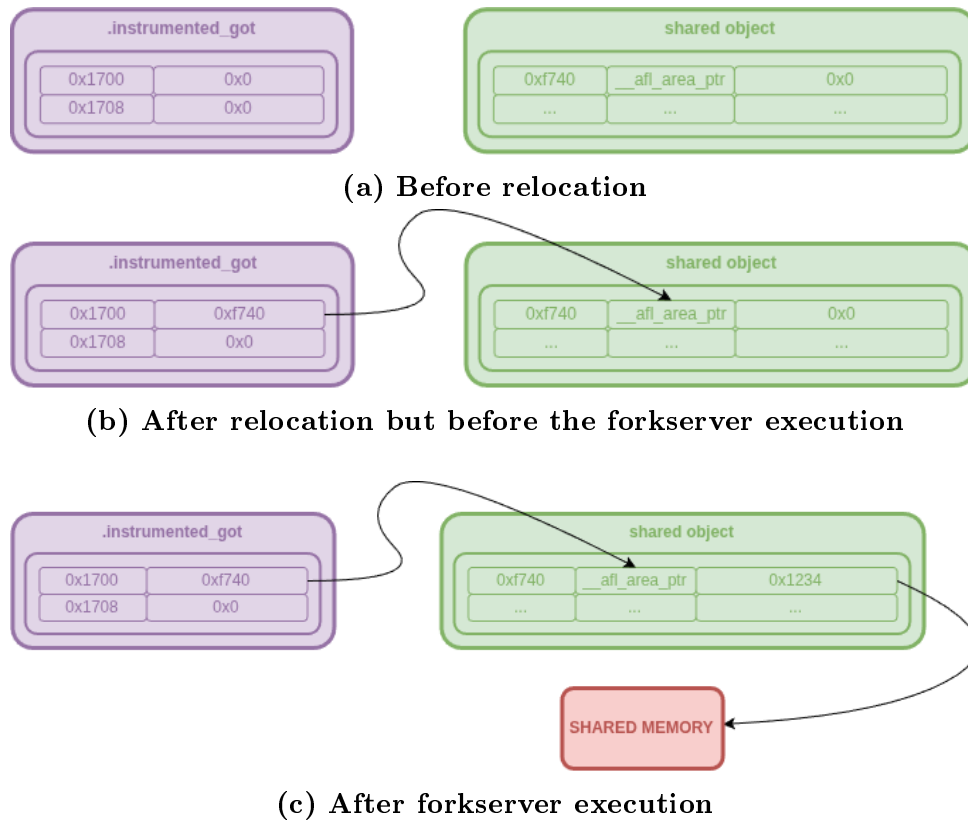


Figure 2.2: Shared Memory relocation between the target binary and the Shared Object

According to fig. 1.5, the relocations are resolved before the forkserver invocation (because the forkserver lies inside the constructor of the shared object). The correct value for the SHM address is obtained by the forkserver, but the symbol is resolved beforehand, and therefore the address of the symbol is written inside the GOT before the variable contains the address of the SHM. When the main starts, the forkserver has already been executed, and dereferencing the relocated symbol will result in the correct value initialized by the forkserver. The process can be seen in fig. 2.2. In fig. 2.2a the GOT contains the address `0x0`, because the relocation has not yet been performed (assume this is after the SO loading but before symbol relocation). Just after the relocation of symbols is performed, the GOT will contain the address of the external variable, but that variable will still contain `0x0`, because the SHM has not been attached yet, as shown in fig. 2.2b. After

forkserver execution, the external variable will also contain the address of the SHM, as shown in fig. 2.2c.

2.3. Instrumentation Points Search

At this point the target binary has been modified with:

- A `DT_NEEDED` entry has been added to *.dynamic*, pointing to a shared object that contains the forkserver declared as a constructor, alongside a `RUNPATH` that points to that shared object.
- A symbol representing the pointer to the SHM has been inserted into *.dynsym*.
- A new section that will be used as a personalized GOT is added.
- A relocation that links the symbol to be resolved to the new section is added.

Now the IPs must be identified so that they can subsequently be instrumented. First, a distinction must be made between two types of instrumentation:

- An initial instrumentation is needed in order to initialize some variables needed for the SHM update rule.
- All the other instrumentations perform the actual SHM update.

In the first case, the IP must be as close as possible to the `main` function of the binary. Finding the main address is easy for both stripped and non-stripped binaries, so, once the main address is identified, the tool tries to find the first address after the function preamble. This goal is not always achieved, because there is no unified grammar for the function prologue. It depends on how many registers the callee wants to save on the stack, how many variables the specific function needs ... In general, the first IP lies in the first BB of the main function.

All the other IPs are found with a Depth First Search (DFS) starting from the initial main BB, searching among the successor BBs of the current one. Specifically, once a BB is found and has not already been analyzed or discovered before, it is added to a queue. The IPs discovery procedure can be seen in algorithm 2.1.

Two different lists must be used in order to distinguish between found basic blocks and basic blocks to be instrumented. For example, if a basic block ends with a call, it is “useless” to instrument the next one. In fact, a `call` is like a branch that is always taken. If an edge that is always taken is instrumented, this would not add any information for a fuzzer. In fact, if, during fuzzing, the first basic block is discovered, the second will surely be discovered as well. The situation is different for the `jmp` instruction. In fact,

Algorithm 2.1 Instrumentation point search with DFS algorithm

```

1: found  $\leftarrow []$  {found is a list of discovered basic blocks. It contains both the basic blocks
   to be instrumented and the basic blocks for which instrumentation is not required}
2: to_instrument  $\leftarrow []$  {this is the list of basic blocks to be instrumented. It is a subset of
   the “found” list}
3: current = main_bb
4: stack  $\leftarrow []$ 
5: successors  $\leftarrow$  EXTRACTSUCCESSORS(current, found)
6: for s  $\in$  successors do
7:   stack.PUSH((current, s))
8: end for
9: while stack is not empty do
10:  current_edge  $\leftarrow$  stack.POP()
11:  prev  $\leftarrow$  current_edge[0], curr  $\leftarrow$  current_edge[1]
12:  if prev.last_instruction._opcode  $\neq$  “call” then
13:    to_instrument.APPEND(curr) {If the previous basic block ends with a call instruction,
     the current basic block is not instrumented}
14:  end if
15:  found.APPEND(curr)
16:  successors  $\leftarrow$  EXTRACTSUCCESSORS(current, found)
17:  for s  $\in$  successors do
18:    stack.PUSH((current, s))
19:  end for
20: end while

```

although this is also a branch that is always taken, it can be useful, for example in loops, to disambiguate between the backward edge (returning to the beginning of the loop) and the entry edge. An example of this can be seen in fig. 2.3.

The EXTRACTSUCCESSORS function will be discussed in the next chapter, where the implementation details will be covered in greater depth. For the moment, it can be said that this function filters the successors of the current basic block, discarding those that it would make no sense to consider. For example, if at a certain point the binary performs a call to a library function (e.g. `printf`), without excluding this case the program would also instrument the *plt* stub of the library function.

In fig. 2.3 a disassembled CFG obtained with Cutter [22], representing a program that computes the power operation, can be seen. With respect to fig. 2.3a, the IPs found are:

- **0x11c2**: initial main BB. This is where the initial instrumentation will be placed.
- **0x11f1** (*prev*=0x11c2,*next*=0x11f1)
- **0x11db** (*prev*=0x11c2,*next*=0x11db)
- **0x1253**

- (prev=*0x11f1*,next=*0x1253*)
- (prev=*0x11db*,next=*0x1253*)

Apart from the call to the `my_pow` function, all the other functions are exported from `libc`, and so do not need to be instrumented.

With respect to fig. 2.3b, the IPs found are:

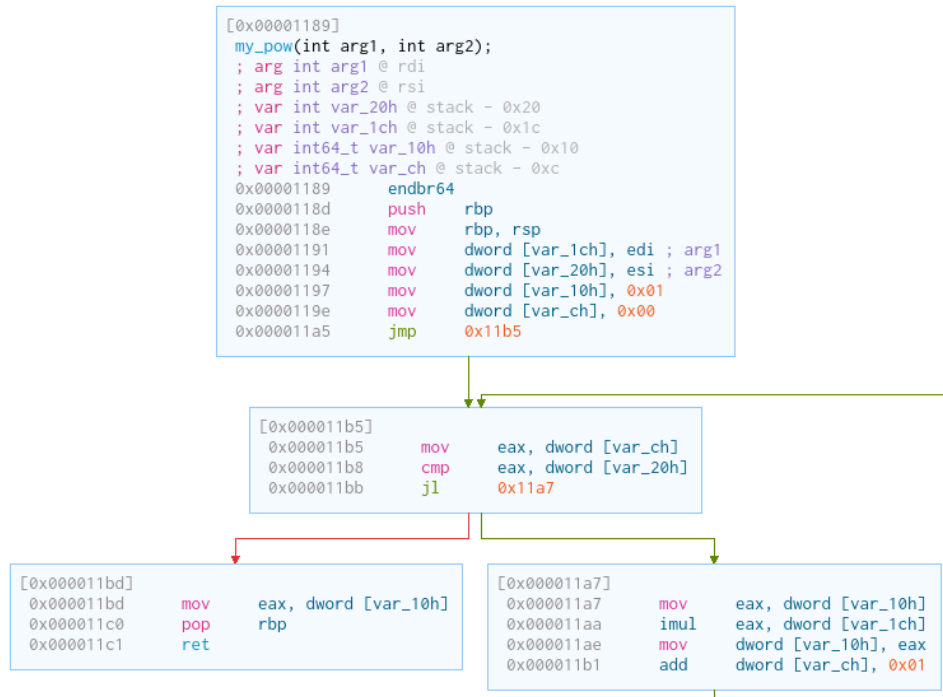
- ***0x11b5***
 - (prev=*0x1189*,next=*0x11b5*)
 - (prev=*0x11a7*,next=*0x11b5*)
- ***0x11a7*** (prev=*0x11b5*,next=*0x11a7*)
- ***0x11bd*** (prev=*0x11b5*,next=*0x11bd*)

The initial BB of the function is not instrumented.

2.4. Actual Instrumentation



(a) Main CFG



(b) Pow function CFG

Figure 2.3: Example of the CFG of a program that computes the power operation between two numbers

Bibliography

- [1] AFL++ contributors. AFL++ Frida Mode. https://github.com/AFLplusplus/AFLplusplus/tree/stable/frida_mode, 2026. Accessed: 2026-09-13.
- [2] AFL++ Developers. Fuzzing binary-only targets. URL https://aflplusplus/docs/fuzzing_binary-only_targets/.
- [3] D. Andriesse, X. Chen, V. van der Veen, A. Slowinska, and H. Bos. An in-depth analysis of disassembly on full-scale x86/x64 binaries. In *25th USENIX Security Symposium (USENIX Security 16)*, pages 583–600, Austin, TX, 2016. USENIX Association. ISBN 978-1-931971-32-4. URL <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/andriesse>.
- [4] D. Bruening, T. Garnett, and S. Amarasinghe. An infrastructure for adaptive dynamic optimization. In *International Symposium on Code Generation and Optimization, 2003. CGO 2003.*, pages 265–275, 2003. doi: 10.1109/CGO.2003.1191551.
- [5] B. Buck and J. K. Hollingsworth. An api for runtime code patching. *The International Journal of High Performance Computing Applications*, 14(4):317–329, 2000. doi: 10.1177/109434200001400404. URL <https://doi.org/10.1177/109434200001400404>.
- [6] M. Böhme, V.-T. Pham, and A. Roychoudhury. Coverage-based greybox fuzzing as markov chain. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS16, pages 1032–1043. ACM, Oct. 2016. doi: 10.1145/2976749.2978428.
- [7] S. Dinesh, N. Burow, D. Xu, and M. Payer. Retrowrite: Statically instrumenting cots binaries for fuzzing and sanitization. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1497–1511, 2020. doi: 10.1109/SP40000.2020.00009.
- [8] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse. AFL++ : Combining incremental steps of fuzzing research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association, Aug. 2020. URL <https://www.usenix.org/conference/woot20/presentation/fioraldi>.

- [9] A. Fioraldi, D. C. Maier, D. Zhang, and D. Balzarotti. Libafl: A framework to build modular and reusable fuzzers. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 1051–1065, 2022.
- [10] S. Gan, C. Zhang, X. Qin, X. Tu, K. Li, Z. Pei, and Z. Chen. Collafl: Path sensitive fuzzing. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 679–696, 2018. doi: 10.1109/SP.2018.00040.
- [11] P. Godefroid, M. Y. Levin, and D. Molnar. Sage: whitebox fuzzing for security testing. *Communications of the ACM*, 55(3):40–44, Mar. 2012. ISSN 1557-7317. doi: 10.1145/2093548.2093564.
- [12] M. Heuse. Afl-dyninst, 2018. URL: <https://github.com/vanhauser-thc/afl-dyninst>.
- [13] M. Heuse. AFL-DynamoRIO. <https://github.com/vanhauser-thc/afl-dynamorio>, 2018. Accessed: 2026-09-13.
- [14] M. Heuse. AFL-PIN. <https://github.com/vanhauser-thc/afl-pin>, 2018. Accessed: 2026-09-13.
- [15] LLVM Project. libfuzzer – a library for coverage-guided fuzz testing. <https://llvm.org/docs/LibFuzzer.html>. Accessed: 2026-09-11.
- [16] C.-K. Luk, R. Cohn, R. Muth, H. Patil, A. Klauser, G. Lowney, S. Wallace, V. J. Reddi, and K. Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. *SIGPLAN Not.*, 40(6):190–200, June 2005. ISSN 0362-1340. doi: 10.1145/1064978.1065034. URL <https://doi.org/10.1145/1064978.1065034>.
- [17] S. Malliserry and Y.-S. Wu. Demystify the fuzzing methods: A comprehensive survey. *ACM Computing Surveys*, 56(3):1–38, Oct. 2023. ISSN 1557-7341. doi: 10.1145/3623375.
- [18] Marsman1996. Missing some instrumentations when instrumenting binary with AFL. <https://github.com/HexHive/retrowrite/issues/28>, Aug. 2021. GitHub Issue #28, HexHive/retrowrite repository. Accessed: 2026-09-13.
- [19] B. P. Miller, L. Fredriksen, and B. So. An empirical study of the reliability of unix utilities. *Communications of the ACM*, 33(12):32–44, Dec. 1990. ISSN 1557-7317. doi: 10.1145/96267.96279.
- [20] S. Nagy, A. Nguyen-Tuong, J. D. Hiser, J. W. Davidson, and M. Hicks. Breaking through binaries: Compiler-quality instrumentation for better binary-only fuzzing.

- In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1683–1700. USENIX Association, Aug. 2021. ISBN 978-1-939133-24-3. URL <https://www.usenix.org/conference/usenixsecurity21/presentation/nagy>.
- [21] O. A. V. Ravnås. Frida: Dynamic instrumentation toolkit for developers, reverse-engineers, and security researchers, 2026. URL <https://github.com/frida/frida>.
- [22] RizinOrg. Cutter, 2026. URL <https://github.com/rizinorg/cutter>. Accessed September 27, 2026.
- [23] M. Wenzl, G. Merzdovnik, J. Ullrich, and E. Weippl. From hack to elaborate technique—a survey on binary rewriting. 52(3), 2019. ISSN 0360-0300. doi: 10.1145/3316415. URL <https://doi.org/10.1145/3316415>.
- [24] J. Xiao, P. Jiang, Z. Zhao, R. Huang, J. Liu, and D. Li. Robust, efficient, and widely available greybox fuzzing for COTS binaries with system call pattern feedback. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 6239–6258, Seattle, WA, Aug. 2025. USENIX Association. ISBN 978-1-939133-52-6. URL <https://www.usenix.org/conference/usenixsecurity25/presentation/xiao-jifan>.
- [25] M. Zalewski. Technical "whitepaper" for afl-fuzz. https://github.com/google/AFL/blob/master/docs/technical_details.txt. Accessed: 2026-09-19.
- [26] M. Zalewski. American Fuzzy Lop. <http://lcamtuf.coredump.cx/afl/>, 2017. Accessed: 2026-09-13.
- [27] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang. Fuzzing: A survey for roadmap. *ACM Computing Surveys*, 54(11s):1–36, Jan. 2022. ISSN 1557-7341. doi: 10.1145/3512345.

List of Acronyms

BB Basic Block. 2, 4, 5, 15–17, 20, 26–28

CFG Control Flow Graph. 2, 4, 17, 27

DBI Dynamic Binary Instrumentation. 3, 4, 14–16, 19

DFS Depth First Search. 26

ELF Executable and Linkable Format. 7, 8, 11, 12

GOT Global Offset Table. 8, 9, 11, 13, 24–26

IP Instrumentation Point. 4–6, 26–28

IR Intermediate Representation. 6, 16

JIT Just in-time. 15

PLT Procedure Linkage Table. 8, 9, 11

SBI Static Binary Instrumentation. 2, 14–16, 19

SHM Shared Memory. 16, 17, 20, 24–26

SO Shared Object. 8–12, 23–25

TB Translation Block. 15

VMA Virtual Memory Area. 23

List of Figures

1.1	Generic diagram describing the Coverage-Guided Grey-box fuzzing workflow	3
1.2	An example of a Static Binary Rewriting Workflow	5
1.3	An example of Static Instrumentation using the trampoline technique . . .	6
1.4	An example of Lazy Binding involving .plt and .got	10
1.5	Dynamically linked ELF loading procedure	11
1.6	Example of a <i>.dynsym</i> table for a shared object	12
1.7	Example of a <i>.rela.dyn</i> relocation table extracted from a binary	12
1.8	Do-while CFG	17
1.9	AFL++ Forkserver Protocol	18
2.1	An example of ELF execution with dynamic dependencies and a constructor function	23
2.2	Shared Memory relocation between the target binary and the Shared Object	25
2.3	Example of the CFG of a program that computes the power operation between two numbers	29

List of Tables

List of Algorithms

1.1	Relocation resolution at ELF load time	13
1.2	RESOLVESYMBOL function used by algorithm 1.1	14
1.3	AFL-style SHM update rule	17
1.4	Do-While Example	17
2.1	Instrumentation point search with DFS algorithm	27

